Conmutación de paquetes

Capítulo 10 · Comunicaciones y redes de computadores

Fundamentos de Redes I · Universidad Nacional Jorge Basadre Grohmann

Texto base: W. Stallings

- **Jhosep Rodrigo Arocutipa Mamani 2024-119027**
- **Brayhan Deyvid Quispe Cama 2024-119050**
- **Diego Jeancarlos Ramos Marca 2024-119016**
- **Jhonel Apaza Pacompia 2024-119026**
- **Fabricio Renato Monroy Fuentes 2024-119005**



AGENDA

Contenido

01	**Principios y técnicas**	Paquetes, datagramas y circuitos virtuales
02	**Funcionamiento externo e interno**	Cómo se combinan datagrama y circuito virtual
03	**Encaminamiento**	Requisitos, criterios, estrategias y el caso ARPANET
04	**Algoritmos de mínimo coste**	Teoría de Dijkstra y Bellman-Ford y un ejercicio resuelto
05	**X.25**	La interfaz estándar con redes de conmutación de paquetes

Del circuito al paquete



Línea ocupada, casi sin uso

En una conexión usuario/estación típica la línea está desocupada la mayor parte del tiempo: la conmutación de circuitos resulta ineficiente para datos.



Velocidad constante

Los dos dispositivos deben transmitir y recibir a la misma velocidad, lo que limita la interconexión de equipos distintos.



La respuesta: conmutación de paquetes

- Los datos viajan en paquetes cortos (límite típico: 1.000 octetos).
- Un mensaje largo se segmenta en varios paquetes.
- Cada paquete lleva datos de usuario más información de control para encaminarlo.
- En cada nodo el paquete se recibe, se almacena y se reenvía al siguiente.

Cómo viaja un mensaje por la red



Cada paquete = cabecera (información de control) + parte de los datos de usuario

Ejemplo: A envía a E. El nodo 4 elige el nodo 5 y deja el paquete en la cola del enlace 4-5.



recibe · almacena · pone en cola · reenvía

Ventajas sobre la conmutación de circuitos



Mejor uso de la línea

Un mismo enlace se comparte en el tiempo entre varios paquetes; en circuitos, la capacidad se reserva y puede quedar vacía.



Conversión de velocidad

Dos estaciones con velocidades distintas intercambian paquetes: cada una se conecta a su nodo a su propia velocidad.



Sin bloqueo de llamadas

Con carga alta, una red de circuitos rechaza conexiones nuevas; una de paquetes sigue aceptando y solo aumenta el retardo.

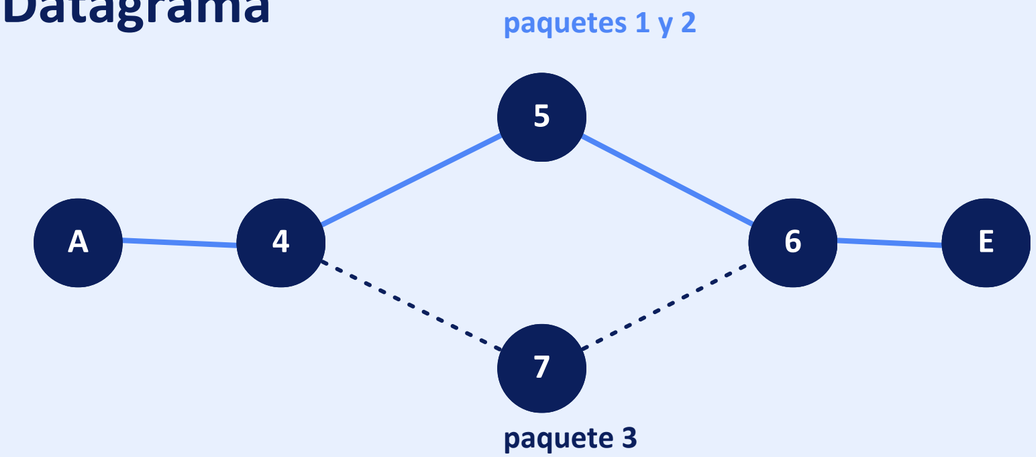


Prioridades

Si un nodo tiene varios paquetes en cola, envía primero los de mayor prioridad, que sufren menos retardo.

Datagrama o circuito virtual

Datagrama



- Cada paquete se trata de forma independiente y lleva la dirección de destino.
- El nodo 4 decide según sus colas: los paquetes 1 y 2 van por el nodo 5; el 3, por el 7.
- Pueden llegar desordenados o perderse: E debe reordenar y detectar pérdidas.

Circuito virtual



- Antes de los datos se fija la ruta: Petición de llamada y Llamada aceptada.
- Cada paquete lleva un identificador de circuito virtual, no la dirección de destino.
- Todos siguen el camino 4-5-6 sin decidir ruta por paquete; se cierra con Petición de liberación.

¿Cuándo conviene cada técnica?

Circuito virtual

- La red da orden secuencial y control de errores.
- Los paquetes viajan más rápido: la ruta se decide una sola vez.
- Atractivo para intercambios largos, como transferir ficheros o acceder a terminales remotos.

Datagrama

- Sin fase de establecimiento: más rápido para uno o pocos paquetes.
- Más flexible: puede rodear una zona congestionada.
- Más seguro ante fallos: si un nodo cae, hay rutas alternativas; en circuitos virtuales se pierden los que lo atraviesan.

En la práctica, casi todas las redes de paquetes usan circuitos virtuales por dentro, por razones históricas y de fiabilidad.

Circuitos frente a paquetes

	Conmutación de circuitos	Paquetes: datagramas	Paquetes: circuitos virtuales
Ruta	Dedicada, para toda la conversación	No dedicada, se decide por paquete	No dedicada, se fija para toda la conversación
Transmisión	Continua; los mensajes no se almacenan	Por paquetes; se almacenan hasta su envío	Por paquetes; se almacenan hasta su envío
Retardo	Establecer la llamada; transmisión despreciable	Retardo de transmisión de paquetes	Establecer la llamada y transmitir paquetes
Sobrecarga	Puede bloquear el establecimiento de llamadas	Aumenta el retardo del paquete	Puede bloquear llamadas y aumenta el retardo
Pérdidas	El usuario se protege	La red, por paquete individual	La red, por secuencia de paquetes
Velocidad y código	Sin conversión	Con conversión	Con conversión
Ancho de banda	Fijo; sin bits extra tras establecer	Dinámico; bits extra en cada paquete	Dinámico; bits extra en cada paquete

Fuente: Tabla 10.1 del capítulo (resumida).

Funcionamiento externo e interno

	Interno: circuito virtual	Interno: datagrama
Externo: circuito virtual	Se crea una ruta dedicada y todos los paquetes siguen el mismo camino.	La red trata cada paquete por separado y, si hace falta, los ordena en el nodo de destino antes de entregarlos.
Externo: datagrama	El usuario no ve conexión, pero la red mantiene una conexión lógica entre estaciones para futuros envíos.	Cada paquete es independiente, tanto para el usuario como para la red.

X.25 es el ejemplo clásico de servicio de circuito virtual externo.

Encaminamiento: requisitos y diseño

Requisitos de la función

- Exactitud
- Simplicidad
- Robustez
- Estabilidad
- Imparcialidad
- Optimización
- Eficiencia

La robustez puede traer inestabilidad, y la imparcialidad choca con la optimización.

Criterio de funcionamiento

Número de saltos, coste, retardo o eficiencia.

Instante de decisión

Por paquete (datagramas) o por sesión (circuitos virtuales).

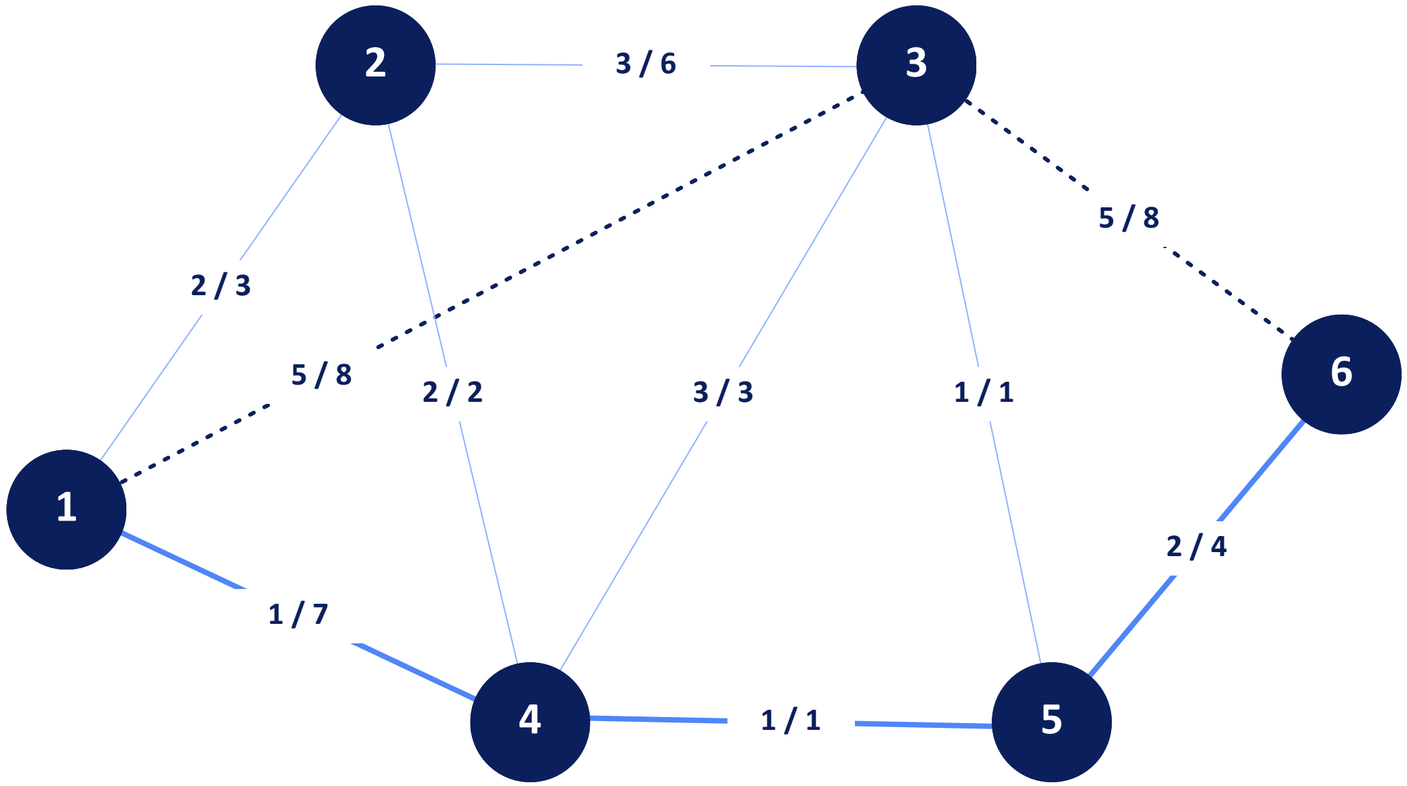
Lugar de decisión

Distribuido (cada nodo), centralizado (nodo central) o en el origen.

Información y actualización

Ninguna, local, de nodos vecinos o de todos; continua, periódica o ante cambios.

Menos saltos frente a mínimo coste



Menos saltos: 1-3-6
Coste 5 + 5 = 10

Mínimo coste: 1-4-5-6
Coste 1 + 1 + 2 = 4

El coste puede depender de la velocidad (más velocidad, menos coste) o del retardo de la cola. Por flexibilidad, el mínimo coste se usa más que el menor número de saltos.

Coste de cada enlace: ida / vuelta (del nodo menor al mayor)

Cuatro estrategias de encaminamiento



Estático

Una ruta fija para cada par origen-destino, guardada en una matriz de encaminamiento.
Simple, pero no reacciona a fallos ni a congestión.



Inundaciones

Cada nodo reenvía el paquete por todos sus enlaces salvo el de llegada.
Muy robusto y sin información de red, pero genera mucho tráfico.



Aleatorio

Un solo enlace de salida elegido al azar, con probabilidad proporcional a su velocidad.
Sencillo y robusto; rutas no óptimas y menos tráfico que inundaciones.



Adaptable

Las rutas cambian con fallos y congestión; los nodos intercambian estado.
La más usada, pero más compleja y con riesgo de oscilaciones.

Encaminamiento estático e inundaciones

Estático: tabla del nodo 1 (Figura 10.6)

Destino	2	3	4	5	6
Siguiente nodo	2	4	4	4	4
Alternativo	4	3	2	3	3

- Basta guardar el siguiente nodo: la ruta 1-4-5-6 se arma salto a salto.
- Los nodos alternativos permiten sortear un nodo o enlace caído.
- Con datagramas o circuitos virtuales da lo mismo: la ruta es siempre la misma.

Inundaciones: cuenta de saltos = 3 (1 → 6)

3
copias tras el 1.er salto

9
copias tras el 2.º salto

22
copias nuevas en el 3.er salto

- El nodo 6 recibe 1 copia en el 2.º salto y 4 más en el 3.º; luego se descartan.
- Cada copia lleva un identificador único y un contador de saltos que baja en cada nodo.
- Prueba todos los caminos: sirve para mensajes críticos o para difundir información a todos los nodos.

Encaminamiento adaptable en ARPANET

1969

Bellman-Ford distribuido

- El retardo se estima con la longitud de la cola del enlace.
- Los vectores de retardo se intercambian con los vecinos cada 128 ms.
- Ignora la velocidad de línea y reacciona lento a la congestión.

1979

Dijkstra con retardo medido

- El retardo se mide con sellos de tiempo y se promedia cada 10 s.
- Los cambios importantes se difunden por inundación.
- Cada nodo recalcula sus rutas con Dijkstra.

1987

Nueva función de coste

- Mismo algoritmo; el coste depende de la utilización suavizada.
- El coste máximo es 3 veces el mínimo.
- Evita oscilaciones cuando la carga es alta.

Oscilación: si todos los nodos actualizan a la vez, el tráfico salta entre los enlaces A y B y ninguno rinde bien.

Notación y costes de enlace

Símbolo	Significado
N	Conjunto de nodos de la red
s	Nodo origen desde el que se calculan las rutas
w(i, j)	Coste del enlace de i a j: 0 si i = j, ∞ si no están conectados, > 0 si lo están
L(n)	Coste de la mejor ruta de s a n encontrada hasta ahora
T	Nodos cuya ruta de mínimo coste ya es definitiva

¿Qué es el coste?

- Si se minimizan los saltos, cada enlace vale 1.
- Suele ser inverso a la capacidad o proporcional a la carga actual.
- Puede ser distinto en cada sentido, por ejemplo la longitud de la cola de cada lado.

El coste de una ruta es la suma de los costes de sus enlaces. El objetivo es hallar, para cada par de nodos, la ruta de menor suma.

Algoritmo de Dijkstra

1 Inicio

T = {s}

L(n) = w(s, n) para cada vecino de s

2 Siguiente nodo

Elegir x ∉ T con L(x) mínimo

y añadirlo a T

3 Actualizar

L(n) = mín[L(n), L(x) + w(x, n)]

para todo n ∉ T

Se repiten los pasos 2 y 3 hasta que T = N (todos los nodos).
L(n): mejor coste conocido de s a n · w(i, j): coste del enlace i → j (∞ si no hay) · T: nodos con ruta definitiva.
Añade nodos en orden de coste creciente y cada nodo necesita conocer toda la topología.

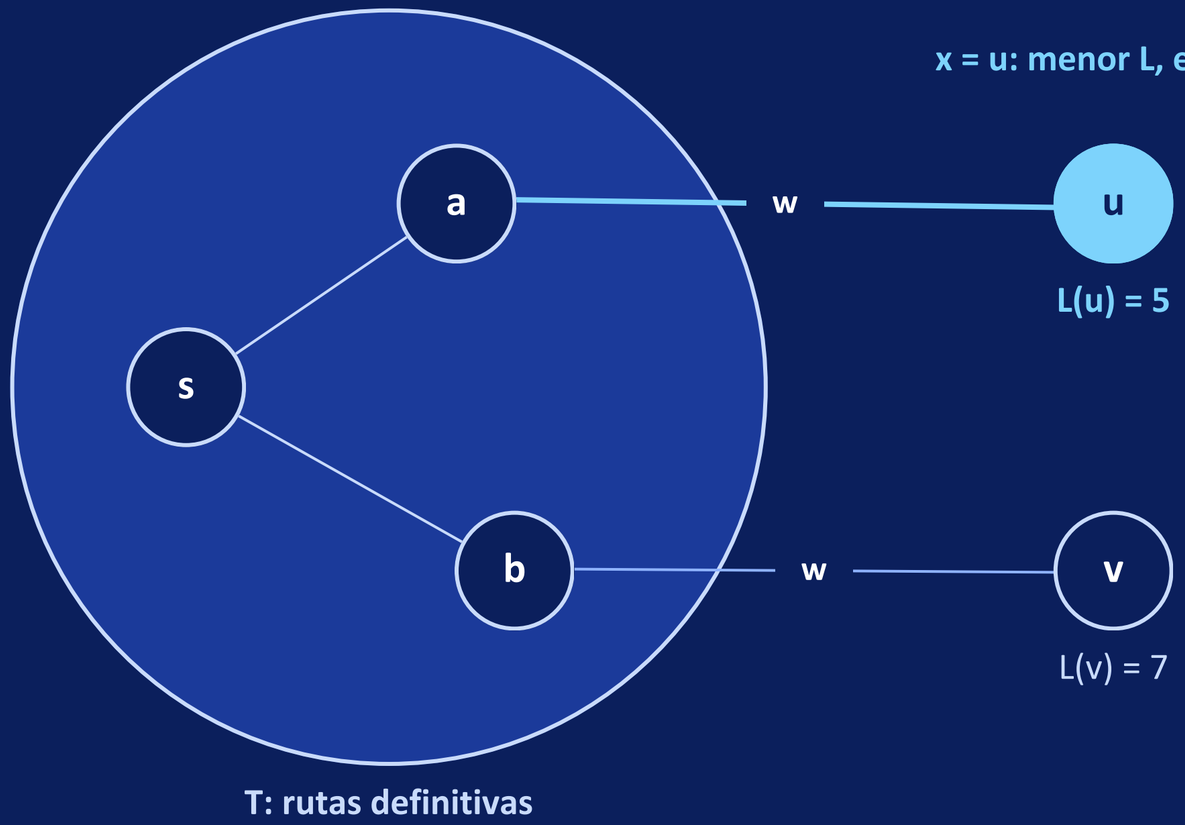
Dijkstra en pseudocódigo

```
T := {s};  L[s] := 0
para cada n distinto de s:  L[n] := w(s, n)
mientras T no contenga todos los nodos:
    x := nodo fuera de T con L[x] mínimo
    añadir x a T
    para cada n fuera de T:
        si L[x] + w(x, n) < L[n]:
            L[n] := L[x] + w(x, n)
            anterior[n] := x
```

Para leerlo

- anterior[n] guarda el nodo previo para reconstruir la ruta al revés.
- Si el menor L[x] es ∞, el resto de nodos no es alcanzable.
- w(i, j) = ∞ cuando no hay enlace directo.

Por qué funciona Dijkstra



Los valores L(u) y L(v) son ilustrativos.

- Tras k iteraciones, T contiene los k nodos más cercanos a s.
- El mejor camino a un nodo fuera de T pasa solo por nodos de T y termina en un enlace directo desde T.
- Por eso basta con elegir el menor L: ningún otro camino puede mejorarlo.
- Exige costes de enlace no negativos.

Dijkstra paso a paso (Figura 10.6, s = 1)

Iter.	T	L(2)	L(3)	L(4)	L(5)	L(6)
1	{1}	2 · 1-2	5 · 1-3	1 · 1-4	∞	∞
2	{1, 4}	2 · 1-2	**4 · 1-4-3**	1 · 1-4	**2 · 1-4-5**	∞
3	{1, 2, 4}	2 · 1-2	4 · 1-4-3	1 · 1-4	2 · 1-4-5	∞
4	{1, 2, 4, 5}	2 · 1-2	**3 · 1-4-5-3**	1 · 1-4	2 · 1-4-5	**4 · 1-4-5-6**
5	{1, 2, 3, 4, 5}	2 · 1-2	3 · 1-4-5-3	1 · 1-4	2 · 1-4-5	4 · 1-4-5-6
6	{1, 2, 3, 4, 5, 6}	2 · 1-2	3 · 1-4-5-3	1 · 1-4	2 · 1-4-5	4 · 1-4-5-6

Cada celda: coste · ruta. Las celdas oscuras muestran un valor que cambió respecto a la fila anterior.

L(3) baja de 5 a 4 y luego a 3 cuando se descubre un camino mejor por los nodos 4 y 5; L(6) aparece recién con el nodo 5.

De las rutas a la tabla de encaminamiento

Árbol de rutas mínimas desde el nodo 1 (Figura 10.21)

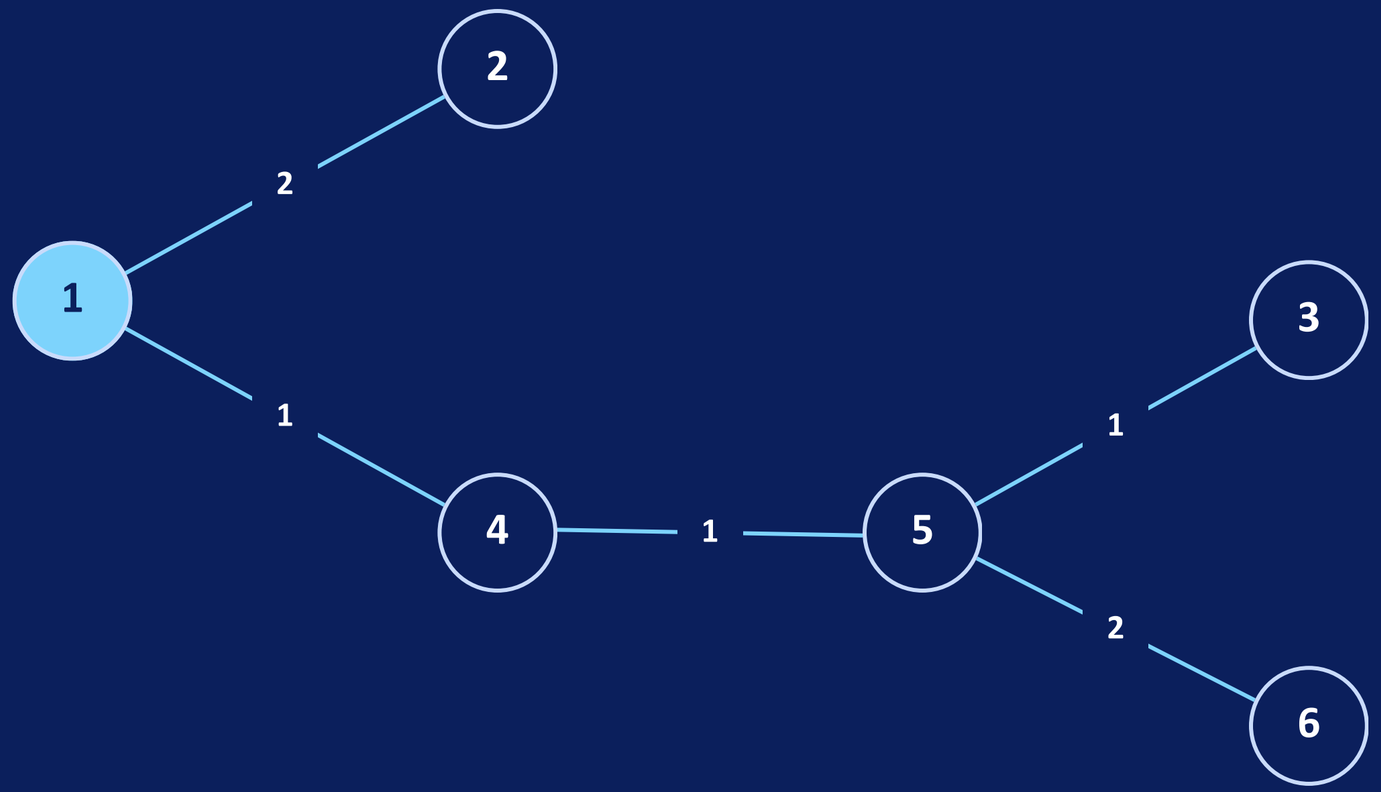


Tabla del nodo 1

Destino	Siguiente	Coste
2	2	2
3	4	3
4	4	1
5	4	2
6	4	4

No hace falta guardar la ruta completa: basta el primer nodo de cada ruta. Después cada nodo repite el cálculo con lo que sigue.

Algoritmo de Bellman-Ford

Idea: buscar primero los caminos de a lo sumo 1 enlace, luego de 2, de 3, etc.

Inicio

$L_0(n) = \infty$ para $n \neq s$; $Lh(s) = 0$

Actualización para cada h

$Lh+1(n) = \text{mín}j\ [\ Lh(j) + w(j, n)\ ]$

Se guarda el predecesor j de mínimo coste. Cada nodo solo necesita los costes de sus enlaces y lo que anuncian sus vecinos.

Figura 10.6 con s = 1 (Tabla 10.4b)

h	Lh(2)	Lh(3)	Lh(4)	Lh(5)	Lh(6)
0	∞	∞	∞	∞	∞
1	2	5	1	∞	∞
2	2	4	1	2	10
3	2	3	1	2	4
4	2	3	1	2	4

Celdas en celeste: valores que mejoraron en esa iteración.

En h = 2 aparece L(6) = 10 por 1-3-6; en h = 3 la ruta 1-4-5-6 lo reduce a 4. Las rutas finales coinciden con las de Dijkstra.

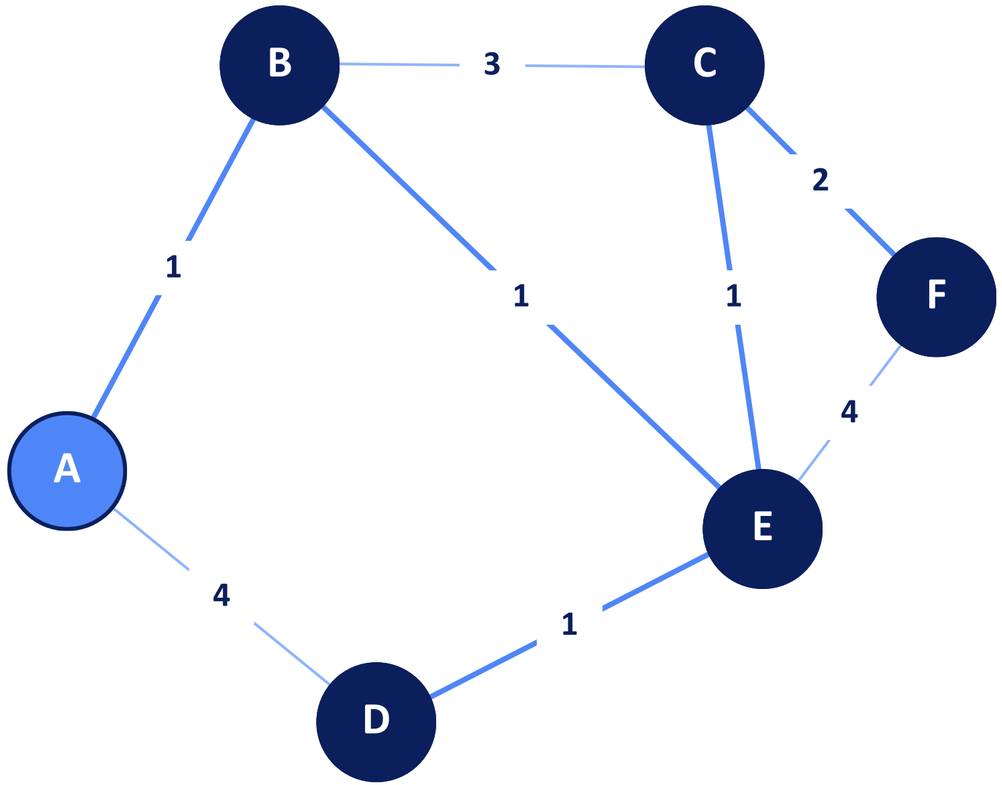
Dijkstra frente a Bellman-Ford

	Dijkstra	Bellman-Ford
Información que necesita	La topología completa: todos los enlaces y sus costes, intercambiados con todos los nodos.	Solo los costes de sus propios enlaces y lo que anuncian sus vecinos directos.
Cómo avanza	Incorpora un nodo por iteración, en orden de coste creciente (fija etiquetas).	Aumenta en uno el máximo de enlaces h en cada iteración (corrige etiquetas).
En ARPANET	Segunda y tercera generación (1979 y 1987).	Primera generación (1969).
Resultado	**Con topología y costes estáticos, ambos convergen a las mismas rutas de mínimo coste.**	

Si el coste depende del tráfico y el tráfico depende de las rutas elegidas, hay realimentación y puede aparecer inestabilidad.

EJERCICIO RESUELTO

Ejercicio: Dijkstra desde A (Fig. 10.20a)



Azul: árbol de rutas mínimas desde A

Iter. · T	B	C	D	E	F
1 · {A}	1	∞	4	∞	∞
2 · {A, B}	1	**4**	4	**2**	∞
3 · {A, B, E}	1	**3**	**3**	2	**6**
4 · {A, B, E, C}	1	3	3	2	**5**
5 · {A, B, E, C, D}	1	3	3	2	5
6 · todos	1	3	3	2	5

Celda: L(n). Oscura = cambió en esa iteración.

Rutas finales desde A
B: A-B (1) · C: A-B-E-C (3) · D: A-B-E-D (3) · E: A-B-E (2) · F: A-B-E-C-F (5)

X.25: la interfaz estándar

3 · Paquete	Servicio de circuito virtual externo entre abonados.
2 · Enlace: LAPB	Transferencia fiable de tramas; deriva de HDLC.
1 · Física: X.21	Interfaz con el enlace (también se usa EIA-232).

DTE ↔ DCE



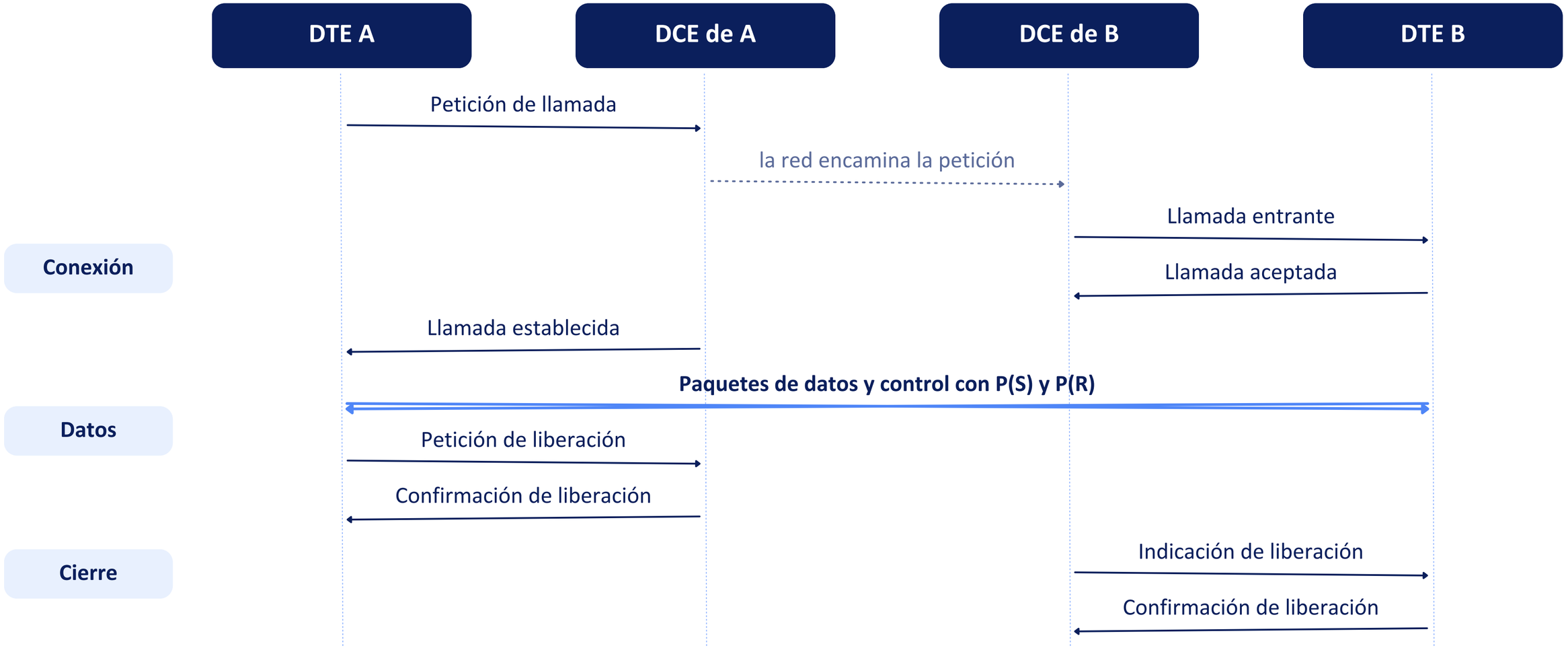
- DTE: equipo terminal de datos (computador o terminal).
- DCE: nodo de conmutación al que se conecta el DTE.

Encapsulamiento



Paquete X.25 = cabecera de nivel 3 + datos · Trama LAPB = paquete + cabecera y cola de enlace

X.25: vida de una llamada virtual



X.25: multiplexación y control de flujo

4.095
circuitos virtuales simultáneos por enlace DTE-DCE

12 bits
para el número de circuito: 4 de grupo y 8 de canal

2
ventana por defecto (hasta 7 o 32.767)

3 bits
numeración por defecto de P(S) y P(R), módulo 8

Multiplexación

- Cada circuito virtual atiende una aplicación, proceso o terminal.
- El número 0 se reserva para diagnóstico; luego van los permanentes, las llamadas entrantes, las bidireccionales y las salientes.
- Separar los rangos evita que DTE y DCE elijan el mismo número a la vez.

Control de flujo y de errores

- Ventana deslizante como en HDLC, con P(S) y P(R); confirmaciones con RR y RNR.
- El rechazo REJ activa un ARQ adelante-atrás-N (go-back-N).
- Bit D: 0 = confirmación local entre DTE y red; 1 = extremo a extremo.

X.25: secuencias y recuperación de errores

Secuencias completas de paquetes



= 0 o más paquetes A + un paquete B

- Paquete A: bit M = 1, bit D = 0 y longitud máxima. Paquete B: cualquier otro.
- La red puede combinar o dividir paquetes sin perder la integridad del bloque; sirve para interconectar redes.
- D = 1 en el paquete B: confirmación extremo a extremo de toda la secuencia.

Reinicio (reset)

Reinicia un circuito virtual: los números de secuencia vuelven a 0 y se pierden los datos en tránsito, que un nivel superior recupera. Causas: pérdida de paquetes, error de secuencia o congestión.

Rearranque (restart)

Error más grave: equivale a liberar todas las llamadas virtuales y reiniciar todos los circuitos. Ejemplo: pérdida temporal del acceso a la red.

Conclusiones

Eficiencia a cambio de complejidad	Los paquetes comparten enlaces y convierten velocidades, pero los nodos nunca conocen el estado de toda la red al instante.
Datagrama o circuito virtual	El datagrama es flexible y robusto; el circuito virtual ofrece orden y control de errores.
Encaminar es elegir una ruta de mínimo coste	Por saltos, retardo o capacidad; las estrategias adaptables son las más usadas.
Dijkstra y Bellman-Ford	Llegan a las mismas rutas; cambia la información que cada nodo necesita.
X.25	Interfaz estándar con circuitos virtuales, multiplexación y control por ventana.

Gracias

